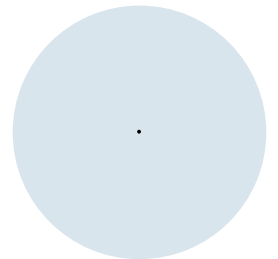


Methoden überschreiben

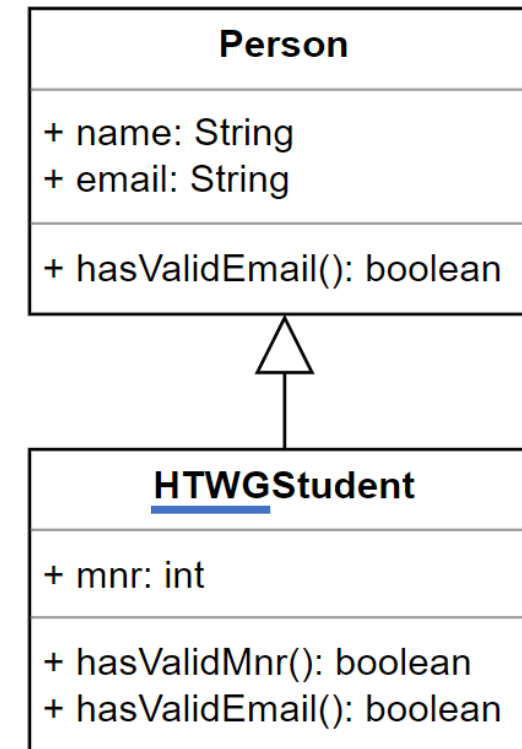


Methoden überschreiben (1)

- Eine Unterklasse kann die Methode einer Oberklasse **überschreiben**, d.h. sie bietet eine eigene Implementierung für diese Methode.
- Beispiel: `hasValidEmail()` für einen **HTWG**-Studenten bedeutet, dass die E-Mail-Adresse nicht nur nicht null sein und ein "@" enthalten, sondern auch auf "htwg-konstanz.de" enden muss.

```
// für Person:  
public boolean hasValidEmail() {  
    return this.email != null  
        && this.email.contains("@");  
}
```

```
// für HTWG-Student:  
public boolean hasValidEmail() {  
    return this.email != null  
        && this.email.contains("@")  
        && this.email.endsWith("htwg-konstanz.de");  
}
```



Beispiel von eben
leicht angepasst!

Methoden überschreiben (2)

```
public class Person {  
    // Attribute und Konstruktor  
    public boolean isValidEmail() {  
        return this.email != null  
            && this.email.contains("@");  
    }  
}
```

```
public class HTWGStudent extends Person {  
    // Attribute, Konstruktor, isValidMnr()  
    public boolean isValidEmail() {  
        return this.email != null  
            && this.email.contains("@")  
            && this.email.endsWith("htwg-konstanz.de");  
    }  
}
```

- die Implementierung von `isValidEmail()` der Unterklasse `HTWGStudent` überschreibt die Implementierung der Oberklasse `Person`:

```
Person max = new Person("Max", "max@example.org");  
System.out.println(max.isValidEmail()); // true  
HTWGStudent maxHTWG = new HTWGStudent("Max", "max@example.org", -1);  
System.out.println(maxHTWG.isValidEmail()); // false
```



code17.
student5
(Package)

Code wiederverwenden

```
public class Person {  
    // Attribute und Konstruktor  
    public boolean hasValidEmail() {  
        return this.email != null  
            && this.email.contains("@");  
    }  
}
```

```
public class HTWGStudent extends Person {  
    // Attribute, Konstruktor, hasValidMnr()  
    public boolean hasValidEmail() {  
        return this.email != null  
            && this.email.contains("@")  
            && this.email.endsWith("htwg-konstanz.de");  
    }  
}
```

- hier steht zweimal dasselbe
- Können wir das nicht wiederverwenden?
 - Also in HTWGStudent die Methode hasValidEmail() von Person aufrufen?

Code wiederverwenden

- Ja, das geht: Mit `super.methode(parameter)` können wir beim Überschreiben einer Methode in einer Unterklasse die Methode der Oberklasse aufrufen und auch deren Rückgabewert auswerten
 - Geht nur über eine Ebene, ein `super.super` ist nicht möglich!

```
public class Person {  
    // Attribute und Konstruktor  
    public boolean isValidEmail() {  
        return this.email != null  
            && this.email.contains("@");  
    }  
}
```

```
public class HTWGStudent extends Person {  
    // Attribute, Konstruktor, isValidMnr()  
    public boolean isValidEmail() {  
        return super.isValidEmail()  
            && this.email.endsWith("htwg-konstanz.de");  
    }  
}
```

- `super.isValidEmail()` ruft den Code der Oberklasse auf
- Lies: "Ein:e HTWG-Student:in hat eine gültige E-Mail-Adresse, wenn er:sie eine gültige E-Mail-Adresse im Sinne einer Person hat und diese außerdem auf "htwg-konstanz.de" endet."



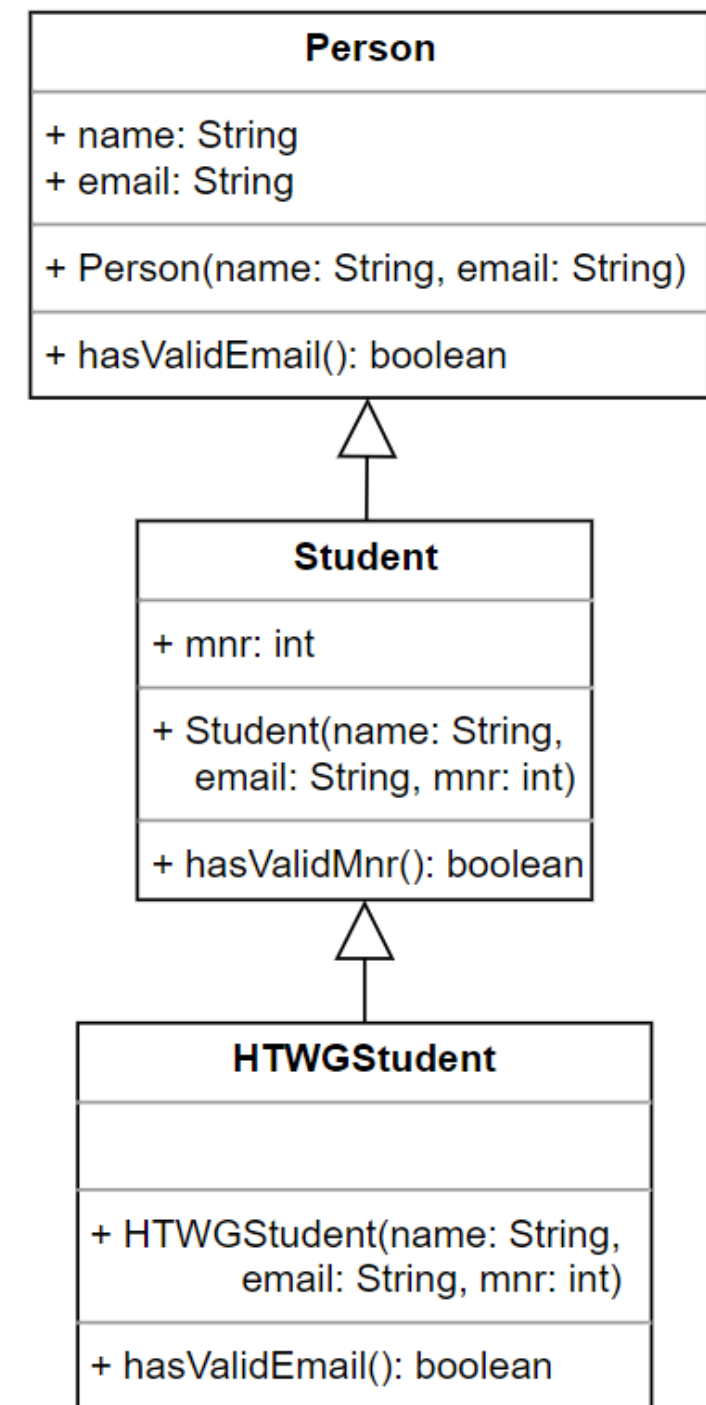
code17.
student6
(Package)

Überladen versus Überschreiben

- Wird gerne verwechselt:
 - **Überladen:** Gleicher Name einer Methode mit **unterschiedlicher Signatur innerhalb einer Klasse**
 - **Überschreiben:** **Unterklasse** definiert Methode mit gleichem Namen, **gleicher Signatur** und gleichem Rückgabewert wie **Oberklasse**

Student-Person-Beispiel komplett

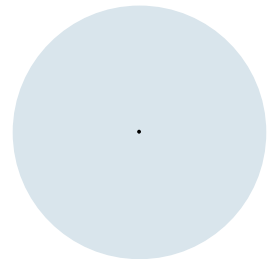
- Da es Student:innen gibt, die keine HTWG-Student:innen sind, ist folgende Struktur sinnvoll
- Hier sind jetzt alle Attribute, Methoden und Konstruktoren aufgeführt
- Fragen an Sie:
 - In welcher Klasse würden Sie eine Methode implementieren, die "Name <E-Mail>" als Adressbucheintrag zurückgibt (z.B. "Beate Beispiel <be123bei@htwg-konstanz.de>")
 - In welcher Klasse eine Methode, welche die letzte Stelle der Matrikelnummer zurückgibt?



Vererbung, Zusammenfassung

- `class Unterklasse extends Oberklasse { ... }`
- Unterklassen ("subclasses") erben Attribute, Konstruktoren und Methoden der Oberklassen ("superclasses")
- Unterklassen können
 - Neue Attribute einführen
 - Neue Methoden einführen
 - Bestehende Methoden neu implementieren
 - Konstruktoren der Oberklasse aufrufen
 - Implementierungen der Oberklassen nutzen
 - (behandeln wir später:) Implementierungen für abstrakte Methoden der Oberklasse bereitstellen
 - ➔ Unterklasse ist dann nicht mehr abstrakt

Sichtbarkeit/Kapselung



Kapselung durch Verstecken

Vorteil:

- Die Implementierung kann lokal geändert werden, ohne dass global alle Stellen, bei denen eine Klasse benutzt wird, angepasst werden müssen

Umsetzung:

- Variablen und interne Methoden verstecken
- Zugriff auf Objekte nur über öffentliche Methoden
- Zugriffsrechte auf eine Klasse
 - durch eigene Klasse ("private")
 - durch abgeleitete Klassen ("protected")
 - durch Klassen desselben Packages ("" – default)
 - durch beliebige Klassen ("public")



code17.
gettersetter
(Package)

Regel 1: Attribute verstecken

- Attribute sollten in der Regel als `private` deklariert werden
 - Dann kann nur innerhalb der Klasse selbst darauf zugegriffen werden
- Zugriff auf Variable über
`getAttribut() {`
 `return this.attribut;`
`} // Getter-Methode`

`setAttribut(Typ neuerWert) {`
 `this.attribut = neuerWert;`
`} // Setter-Methode`

```
public class Address {  
  
    private int plz;  
    // weitere Attribute...  
  
    public int getPlz() {  
        return this.plz;  
    }  
  
    public void setPlz(int plz) {  
        this.plz = plz;  
    }  
}
```



Regel 1: Attribute verstecken

- Attribute sollten in der Regel als `private` deklariert werden
 - Dann kann nur innerhalb der Klasse selbst darauf zugegriffen werden
- Falls in einer geerbten Klasse auf das Attribut zugegriffen werden soll, kann man es als `protected` deklarieren

```
public class Person {  
  
    protected String name;  
    protected String email;  
}
```

```
public class Student extends Person {  
    private int mnr;  
  
    public String toString() {  
        return "Student{" +  
            "name=" + this.name +  
            ", email=" + this.email +  
            ", mnr=" + this.mnr  
            '}';  
    }  
}
```

Zugriff möglich!

Regel 2: Interne Methoden verstecken

- Methoden
 - nur innerhalb der Klasse benutzt: `private`
 - auch von Subklassen benutzt: `protected`
 - von "außen" benutzt: `public`

```
public class Person {  
  
    private String email;  
    // interne Hilfsmethode  
    private boolean emailCorrect(String email){  
        return email.contains("@");  
    }  
    // von "außen" benutzte Methode  
    public void setEmail(String email) {  
        if (!emailCorrect(email)) {  
            throw new IllegalArgumentException();  
        }  
        this.email = email;  
    }  
    //...  
}
```

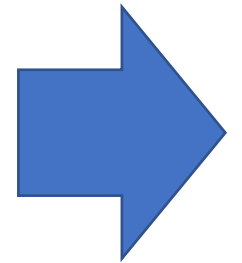
Übliche Regeln/Empfehlungen zusammengefasst

- **Klassen**
 - üblicherweise `public`
 - Ausnahme Hilfsklassen, die nur innerhalb eines Packages verwendet werden: `package private`
- **Attribute**
 - immer `private` (Zugriff über öffentliche get- und set-Methoden, wenn gewünscht)
 - Ausnahme: von Subklassen benutzt, dann `protected`
- **Konstruktoren**
 - üblicherweise `public`
 - Ausnahme: interne Konstruktoren, die nur mittels `this(...)` aufgerufen werden sollen -> `private` oder `protected`, wenn auch von Subklassen benutzt
- **Methoden**
 - nur innerhalb der Klasse benutzt: `private`
 - auch von Subklassen benutzt: `protected`
 - von "außen" benutzt: `public`

Zugriffsmodifikatoren (vereinfacht)

	<i>eigene Klasse</i>	<i>Klassen desselben Pakets</i>	<i>Subklassen</i> ¹	<i>alle anderen Klassen</i>
public	+	+	+	+
protected	+	+	+ ²	-
<i>kein Modifizierer</i>	+	+	-	-
private	+	-	-	-

Auf den folgenden Folien sind alle Fälle noch einmal detailliert (und korrekt) aufgeführt.



¹Subklassen können im selben Package oder in anderen Packages liegen

²bei eigener Instanz und anderen Instanzen der Subklasse

Zugriffsmodifikatoren, public

- **Klassen**, die mit **public** gekennzeichnet sind, können überall (auch in anderen Paketen) genutzt werden*.
- Auf **Attribute**, **Methoden** und **Konstruktoren**, die als **public** vereinbart sind, kann überall dort zugegriffen werden, wo auf die Klasse zugegriffen werden kann.

*Es sei denn, man konfiguriert das anders mit dem ab Java 9 eingeführten Modul-Mechanismus (nicht Teil dieser Vorlesung)



code17.
access_modifiers
(Package)

aus "Dietmar Abts, Grundkurs JAVA"

Zugriffsmodifikatoren, **protected**, **default**, **private**

- Auf **Attribute**, **Methoden** und **Konstruktoren**, die mit **protected** gekennzeichnet sind, können die eigene Klasse und alle anderen Klassen innerhalb **desselben Pakets** zugreifen.
- **protected-Attribute** und **-Methoden** werden an alle Subklassen, die auch anderen Paketen angehören können, weitervererbt und sind dann dort zugänglich.
- **default ("package-private")**: Sind **weder public, protected noch private** angegeben, so können die Klassen, Attribute, Methoden und Konstruktoren in allen Klassen desselben Pakets benutzt werden.
- **Attribute**, **Methoden** und **Konstruktoren**, die mit **private** gekennzeichnet sind, können **nur in der eigenen Klasse** genutzt werden.



code17.
access_modifiers
(Package)

aus "Dietmar Abts, Grundkurs JAVA"

Zugriffsmodifikatoren, Überschreiben

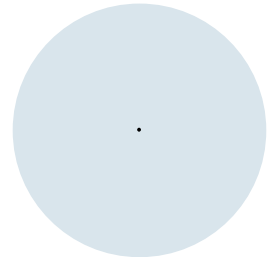
- Beim Überschreiben von Methoden in Subklassen dürfen die Zugriffsrechte nicht reduziert werden. Insbesondere kann also eine public-Methode nur public überschrieben werden. Eine protected-Methode kann mit public oder protected überschrieben werden.



code17.
access_modifiers
(Package)

aus "Dietmar Abts, Grundkurs JAVA"

Typumwandlung und Typüberprüfung



Typumwandlungen, implizit

- Implizite Umwandlung in Oberklasse jederzeit möglich ("upcast").

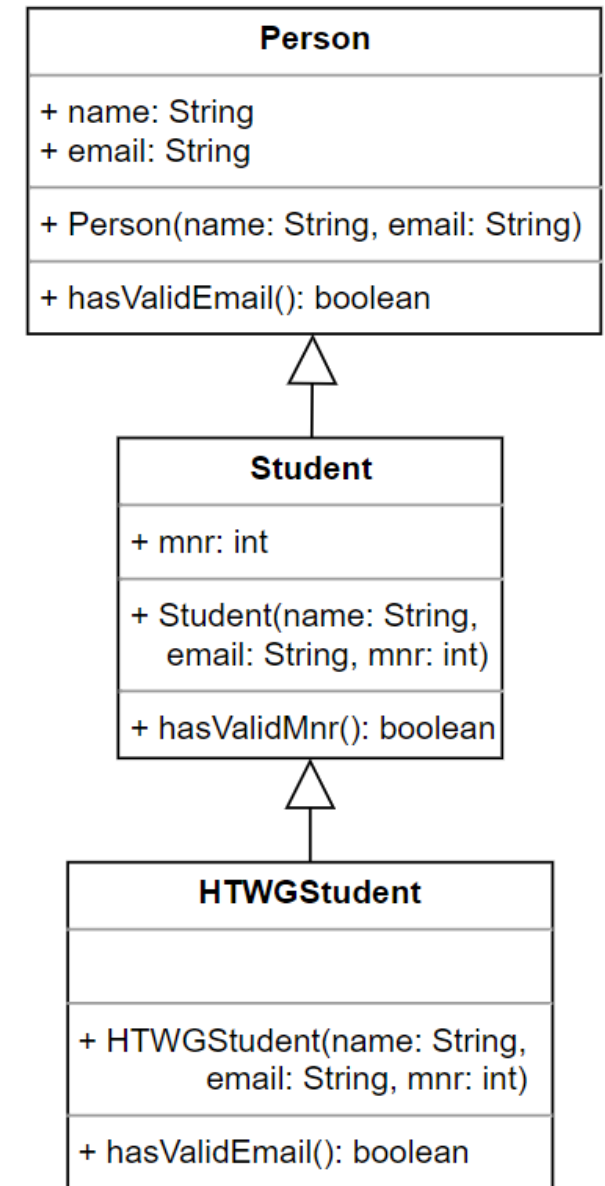
```
Student student = new Student(...);
Person person1 = student;

// oder direkt
Person person2 = new Student(...);

// weiteres Beispiel:
Student student2 = new HTWGStudent(...);
```

- auch über mehrere Ebenen:

```
Person person = new HTWGStudent(...);
```

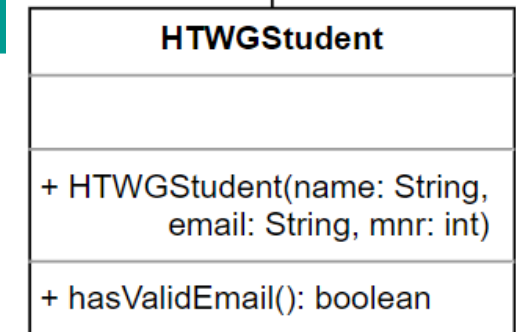
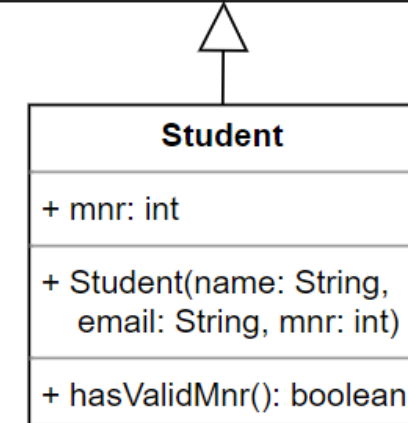
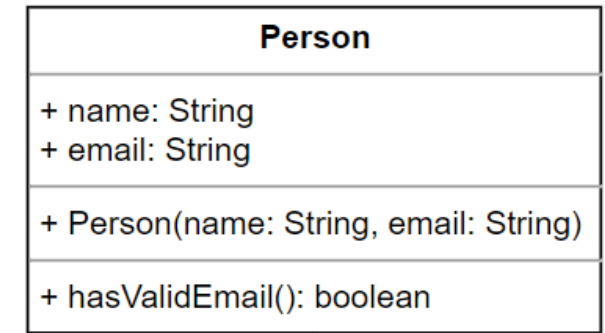


Typumwandlungen, implizit, Anwendung

- Überprüfen, ob alle Personen in einem Array eine gültige E-Mail-Adresse haben:

```
Person max = new Person("Max Muster", "max@example.org");
Person alex = new Person("Alex Schwarz", "alex@mail.org");
Student beate = new Student("Beate Beispiel",
    "bbeisp123@uni-konstanz.de", 123456);
HTWGStudent chris = new HTWGStudent("Chris Potter",
    "ch321pot@htwg-konstanz.de", 654322);

Person[] people = {max, alex, beate, chris}; upcast!
boolean valid = true;
for (Person person : people) {
    valid = valid && person.isValidEmail();
}
System.out.println(valid);
```



upcast



code17.
casting/
ValidEmail

Typumwandlungen, explizit

- Explizite Umwandlung in **Unterklasse** ("downcast") über cast-Operator:

```
Person person = new Student(...);  
Student student = (Student) person;
```

- **Fehler** zur Laufzeit (ClassCastException),
wenn Instanz-Typ nicht passt:

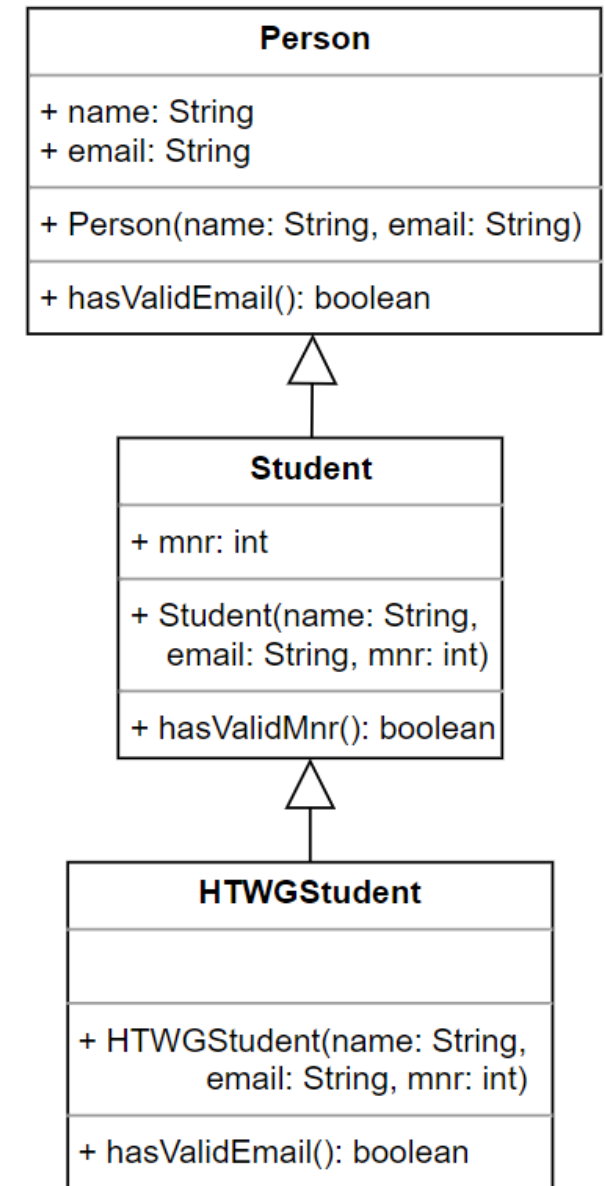
```
Person person = new Person(...);  
// Fehler zu Laufzeit:  
Student student = (Student) person;
```

- **Fehler** zur Compile-Zeit, wenn Typen inkompatibel (Student und String befinden sich nicht in einer Hierarchie):

```
Student student = new Student(...);  
// Fehler zu Compilezeit:  
String string = (String) Student;
```



code17.
casting/
DownCast



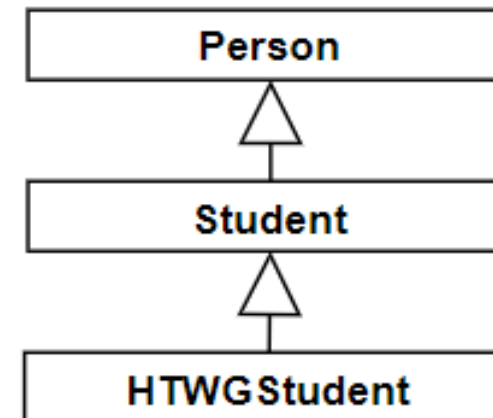
Typüberprüfung

- mit instanceof-Operator kann überprüft werden, ob ein Objekt in einen Typ umgewandelt werden kann

```
Person person = ...;  
System.out.println(person.mnr);  
  
if (person instanceof Student) {  
    Student student = (Student) person;  
    System.out.println(student.mnr);  
}
```

- Wir bekommen irgendwoher eine Instanz von Person (oder einer Unterklasse)
- Zugriff auf die Matrikelnummer ist nicht möglich (da Person keine solche hat)
- Wir überprüfen, ob person eine Student-Instanz ist
- Falls ja, machen wir einen expliziten Downcast
- jetzt können wir auf die Matrikelnummer zugreifen!

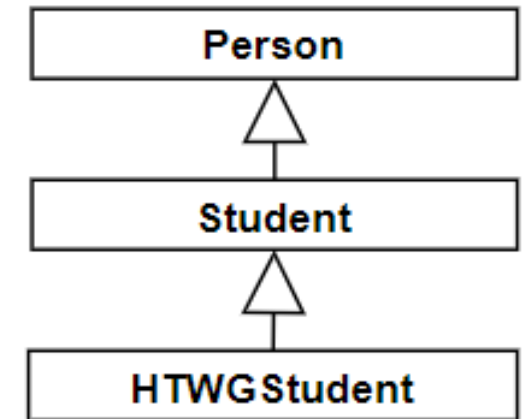
- Überlegen Sie sich: Was passiert im obigen Code
 - wenn person eine Person-Instanz ist?
 - wenn person eine Student-Instanz ist?
 - wenn person eine HTWGStudent-Instanz ist?



Typüberprüfung, Anwendung

- Wollen zählen, wie viele Student:innen unter den Personen in einem Array sind und deren Matrikelnummern ausgeben.

```
Person[] people = {...};
int count = 0;
for (Person person : people) {
    if (person instanceof Student) {
        count++;
        Student student = (Student) person;
        System.out.println(student.mnr);
    }
}
System.out.printf("%d students%n", count);
```



code17.
casting/
Count
Students

Erweitertes instanceof (seit Java 14)

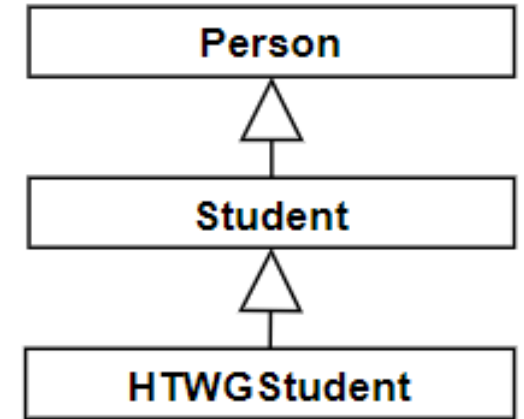
- Typisches Muster: mit instanceof überprüfen und anschließend casten:

```
if (person instanceof Student) {  
    Student student = (Student) person;  
    System.out.println(student.mnr);  
}
```

- geht seit Java 14 einfacher:

```
if (person instanceof Student student) {  
    System.out.println(student.mnr);  
}
```

- überprüft, ob person vom Typ Student ist und stellt, falls ja, eine Variable student vom Typ Student zur Verfügung

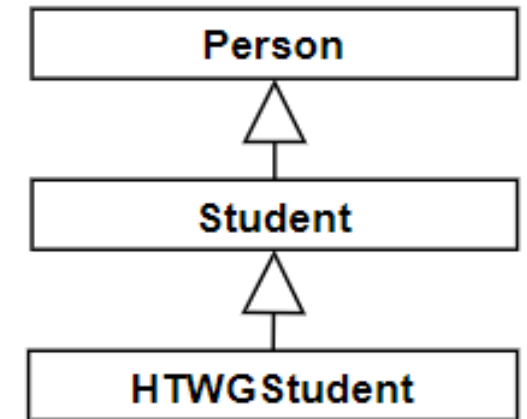


code17.
casting/
InstanceOf2

Erweitertes instanceof, Anwendung

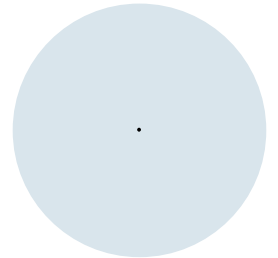
- Beispiel von eben: Wollen zählen, wie viele Student:innen unter den Personen in einem Array sind und deren Matrikelnummern ausgeben.

```
Person[] people = {...};
int count = 0;
for (Person person : people) {
    if (person instanceof Student student) {
        count++;
        System.out.println(student.mnr);
    }
}
System.out.printf("%d students\n", count);
```



code17.
casting/
Count
Students2

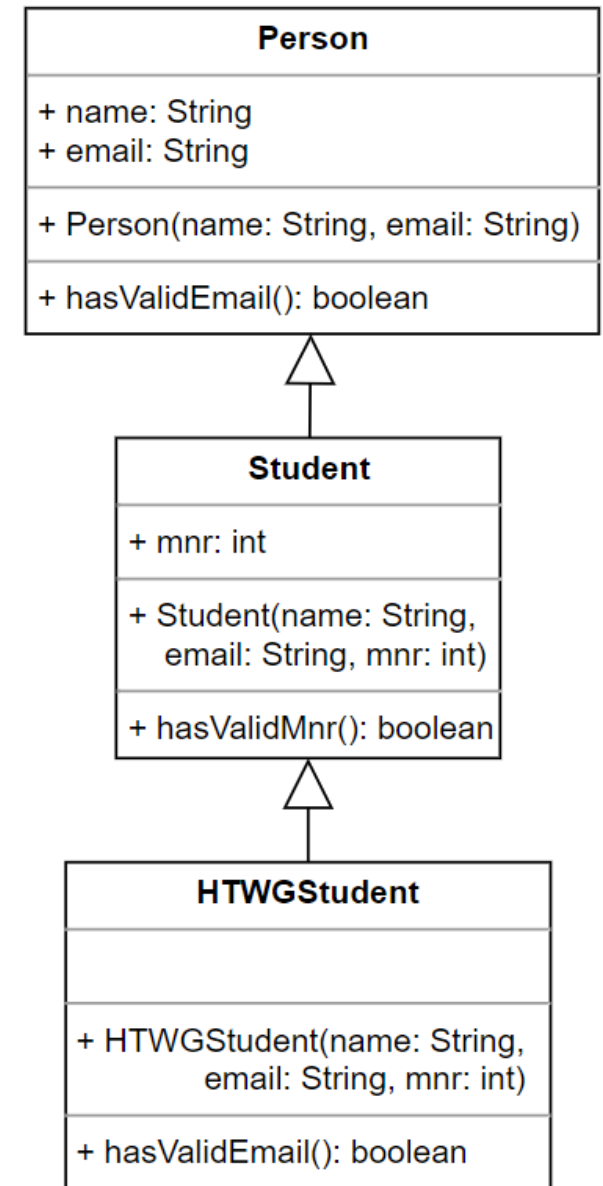
Polymorphie



Wiederholung Typumwandlung

```
Person[] people = {  
    new Person("Alex", "alex@example.org"),  
    new Student("Beate", "bbeisp123@uni-konstanz.de", 123456),  
    new HTWGStudent("Chris", "ch321pot@htwg-konstanz.de", 654322)  
};
```

- das funktioniert, weil Student von Person und HTWGStudent von Student erbt.
- implizite Typumwandlungen ("upcast") finden statt



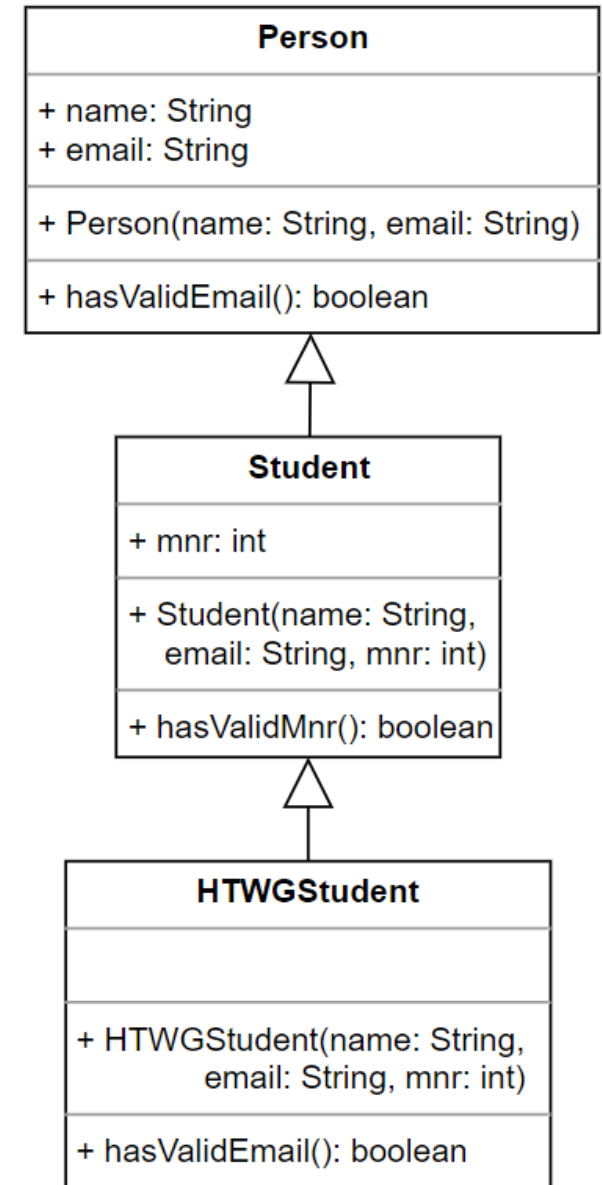
Polymorphie

```
Person[] people = {  
    new Person("Alex", "alex@example.org"),  
    new Student("Beate", "bbeisp123@uni-konstanz.de", 123456),  
    new HTWGStudent("Chris", "ch321pot@htwg-konstanz.de", 654322)  
};  
// random int between 0 and people.length-1  
int randomIndex = (int) (Math.random() * people.length);  
Person random = people[randomIndex];  
  
System.out.println(random.toString());  
System.out.println(random.isValidEmail());
```

- Welche Methoden werden hier aufgerufen?
- Könnten in Person, Student oder HTWGStudent implementiert sein.
- Steht erst zur Laufzeit fest, wenn bekannt ist, von welchem tatsächlichen Typ random ist



code17.
polymorphic/
RandomElement



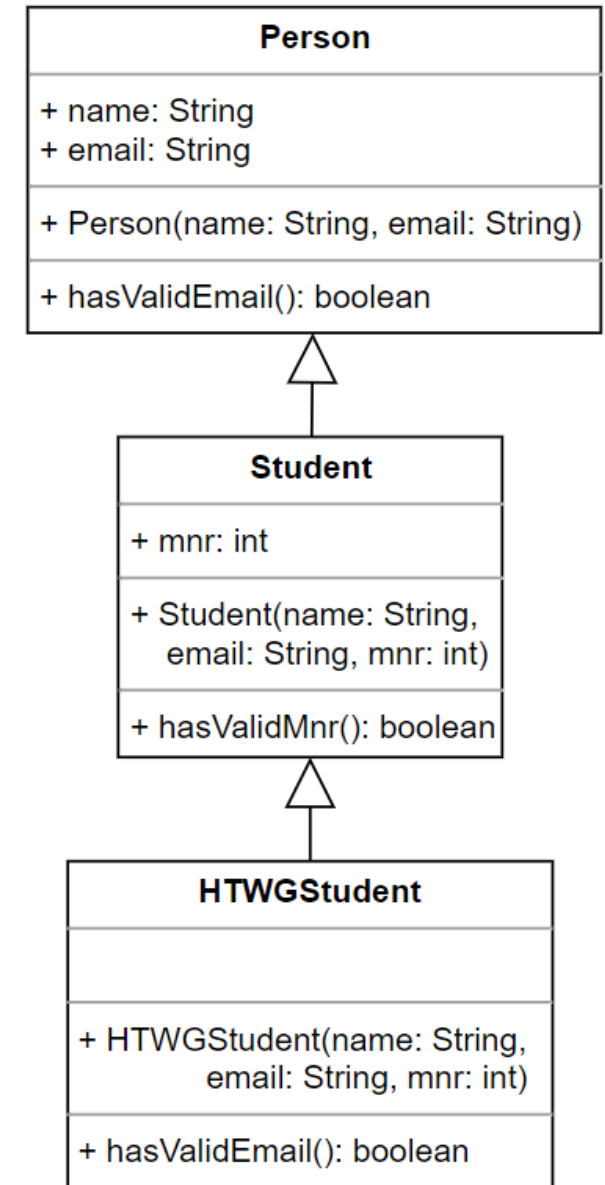
Polymorphie

```
Person[] people = {  
    new Person("Alex", "alex@example.org"),  
    new Student("Beate", "bbeisp123@uni-konstanz.de", 123456),  
    new HTWGStudent("Chris", "ch321pot@htwg-konstanz.de", 654322)  
};  
// random int between 0 and people.length-1  
int randomIndex = (int) (Math.random() * people.length);  
Person random = people[randomIndex];  
  
System.out.println(random.toString());  
System.out.println(random.isValidEmail());
```

- Beispiel `isValidEmail()`:
 - Wenn `random` eine Instanz von `HTWGStudent` ist, wird hier auch überprüft, ob die Adresse mit "htwg-konstanz.de" endet
 - Ansonsten wird die Methode von `Person` aufgerufen (auch im Fall `random` ist Instanz von `Student`)



code17.
polymorphic/
RandomElement



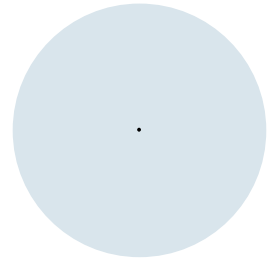
Polymorphie, Zusammenfassung

- Vielgestaltigkeit
- **Variable** eines **Supertyps** kann **Objekt eines Subtyps** referenzieren
 - Subtyp abgeleitet von Supertyp (mittels extends)
 - Im Beispiel Person `person = new Student(...);`
- Deklarierter Typ vs. tatsächlicher Typ
- **Methodenaufruf** bezogen auf **tatsächlichen Typ**

Wozu Vererbung?

- **Code (Implementierung) wiederverwenden**
 - Person und alle Subklassen
 - gemeinsame Attribute Name und E-Mail-Adresse
 - gemeinsame isValidEmail()-Methode (zumindest die "Grundeigenschaften nicht-null und mit @-Zeichen" sind gleich)
 - Student und HTWGStudent
 - gemeinsames Attribut Matrikelnummer
 - gemeinsame Methode isValidMnr()
- **Einheitliche Behandlung unterschiedlicher (vielgestaltiger) Instanzen mit gemeinsamem Basistyp**
 - "Liste" Person[] zur Überprüfung auf gültige Mail-Adressen
 - Bei Methoden-Aufruf toString() oder isValidEmail() wird die konkrete Implementierung der Subklasse benutzt

Interfaces



Abstraktion: Interfaces

- **Klassen** dienen der **Implementierung**
- Interfaces dienen der **Modellierung/Spezifikation**
- **Ein** Interface kann von **verschiedenen** Klassen implementiert werden
 - (Eine Klasse kann auch verschiedene Interfaces implementieren)
- Ziel: **weniger Abhängigkeit** von bestimmten Implementierungen

- Keine Implementierung, nur Signaturvorgabe
- Keine Attribute, kein Konstruktor

Interfaces



```
public interface Summarizable {  
    String getSummary();  
}
```

Interfaces



- Keine Implementierung, nur Signaturvorgabe
- Keine Attribute, kein Konstruktor
- Klassen, die ein Interface implementieren, müssen alle Methoden des Interfaces definieren (Interfaces können mehr als eine Methode definieren)

```
public interface Summarizable {  
    String getSummary();  
}  
  
public class Address implements Summarizable {  
    public String name;  
    public String city;  
    //...  
    public String getSummary() { return this.name  
        + " is living in " + ...; }  
}  
  
public class Person implements Summarizable {  
    public String name;  
    public int age;  
    //...  
    public String getSummary() { return this.name + " is "  
        + age + " years old"; }  
}
```



code17.
summarizable
(Package)

Interfaces, allgemein



code17.
summarizable/
Main1

- Deklaration in eigener Datei <Interfacename>.java
- Implementierung durch Klasse:

```
<Klassenname> implements <Interfacename> {  
    // Alle Methoden implementieren  
}
```



```
public class Address implements Summarizable {  
    // Alle Methoden implementieren  
}
```
- Variablendeklaration auf Basis eines Interfaces

```
<Interfacename> <Variablenname>;           Address address;  
                                           Address address = new Address();
```
- Variablendeklaration auf Basis einer Klasse

```
<Klassenname> <Variablenname>;           Summarizable summarizable = address;
```
- "Interface inheritance", "subtyping": Implizite Typumwandlung Klasse → Interface möglich

```
<Interfacename> <Variablenname> = new <Klassenname>(...);  
Summarizable summarizable = new Address();
```

Interfaces, Summarizable, Vorteile

```
void printSummaryOfAddress(Address address) {  
    System.out.println(address.getSummary());  
}  
  
void printSummaryOfPerson(Person person) {  
    System.out.println(person.getSummary());  
}
```

- Da in beide Methoden nur wichtig ist, dass der Parameter über eine Methode getSummary verfügt, können sie durch eine einzelne Methode ersetzt werden:

```
void printSummary(Summarizable summarizable) {  
    System.out.println(summarizable.getSummary());  
}
```



code17.
summarizable/
Main2a



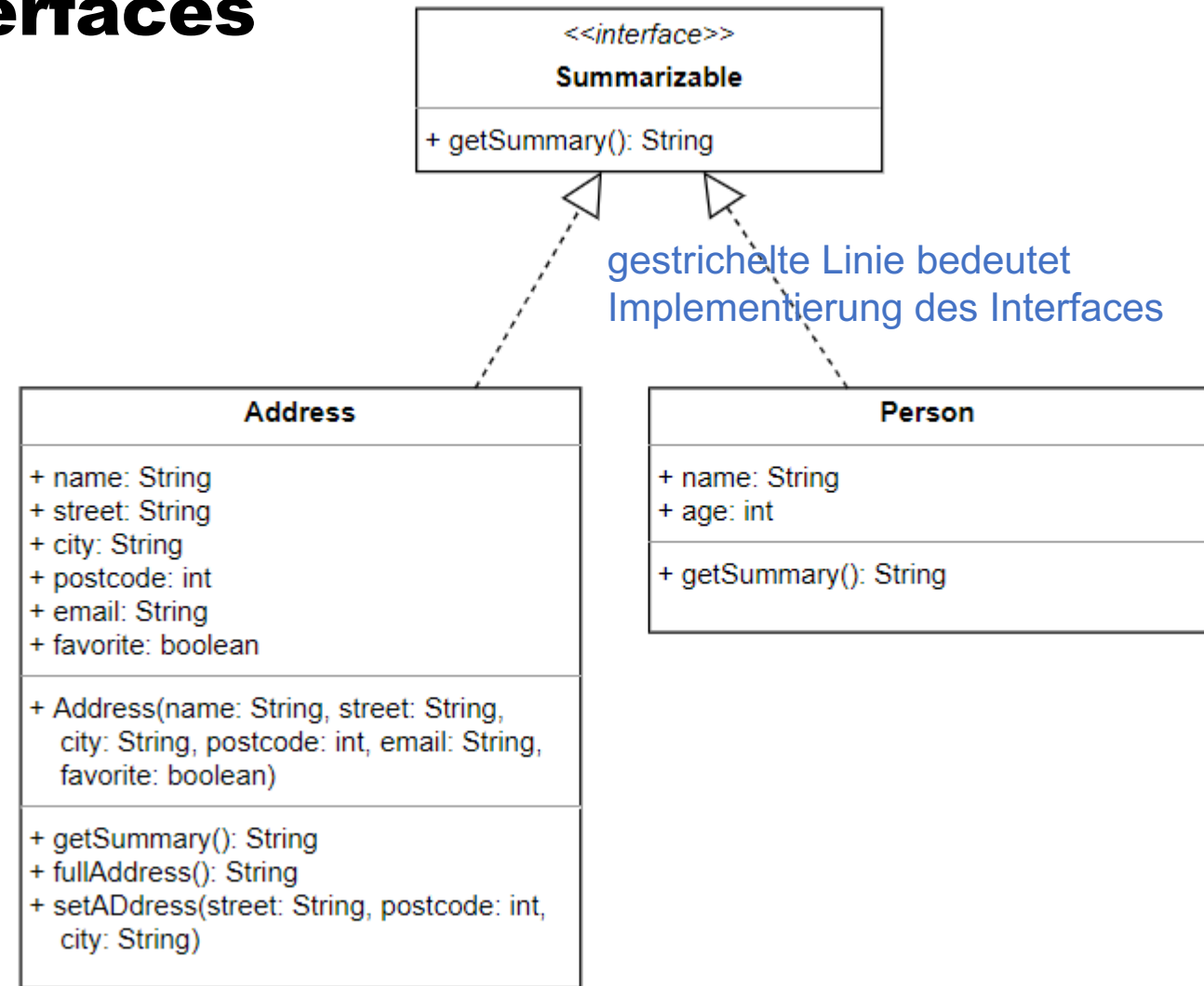
code17.
summarizable/
Main2b

Typumwandlungen bei Interfaces

- Address und Person implementieren Summarizable
- **Impliziter Upcast** möglich

```
Summarizable sum1 = new Address(...);  
// oder  
Summarizable sum2 = new Person();
```

- Hinweis: Vererbung zwischen Address und Person nicht sinnvoll: Weder
 - "Jede Person ist ein Adresse"
 - noch "Jede Adresse ist eine Person"ergeben Sinn.



Interfaces, IConsolePrinter

- weiteres Beispiel: verschiedene Klassen bieten ähnliche Funktionalität (gleiche Methode)

```
public interface IConsolePrinter {  
    void print(String s);  
}  
public class SimpleConsolePrinter implements IConsolePrinter {  
    public void print(String s) {  
        System.out.println(s);  
    }  
}  
public class UpperCaseConsolePrinter implements IConsolePrinter {  
    public void print(String s) {  
        System.out.println(s.toUpperCase());  
    }  
}
```

Im Code-Beispiel gibt es noch eine zweite Methode `void print(int i);`



code17.
ConsolePrinter/
Main1

Interfaces, IConsolePrinter, Vorteile

Situation:

- An verschiedenen Stellen im Code sollen Ausgaben vorgenommen werden
- Formatierung (simple, uppercase, fancy) soll zentral an einer Stelle konfiguriert werden

Lösung:

- statische Methode erzeugt an zentraler Stelle IConsolePrinter-Instanz
- Implementierung kann an dieser Stelle gewählt werden
- Für Verwender von IConsolePrinter ist das transparent (aber auch egal)



code17.
ConsolePrinter/
Main2

Interfaces: Beispiel aus dem Java SDK

```
public interface CharSequence {  
    int length();  
    char charAt(int index);  
    boolean isEmpty();  
    CharSequence subSequence(  
        int start, int end);  
    String toString();  
    // weitere  
}
```

```
public class String  
    implements CharSequence { ... }
```

```
public class StringBuilder  
    implements CharSequence { ... }
```

- Interface für Zeichensequenzen
- wird von String implementiert
- aber auch von [StringBuilder](#), [StringBuffer](#) und weiteren
- abstrahiert von der Klasse String



code17.
Join

Interfaces: Beispiel aus dem Java SDK

```
public interface CharSequence { ... }  
  
public class String implements CharSequence { ... }  
  
public class StringBuilder implements CharSequence { ... }
```

- Methoden, die Zeichensequenzen erwarten, können CharSequence-Instanzen als Parameter verwenden und funktionieren damit auch mit StringBuilder, StringBuffer etc.
- Eine solche Methode ist `String.join(...)`

```
static String    join(CharSequence delimiter,  
                    CharSequence... elements)
```

Returns a new String composed of copies of the CharSequence elements joined together with a copy of the specified delimiter.

Interfaces, Erweiterung

- Interfaces kann man erweitern

```
interface <Interfacename> extends <AndererInterfacename> {  
    // Zusätzliche Konstanten und Methoden  
}
```

- Man kann auf Interfaces casten (Typumwandlung)

```
interface IMyInterface extends IMySimpleInterface { ... }  
class MyClass implements IMyInterface { ... }  
  
IMyInterface myIntImpl = new MyClass();  
IMySimpleInterface mySimpleIntImpl = myIntImpl; // up-cast
```

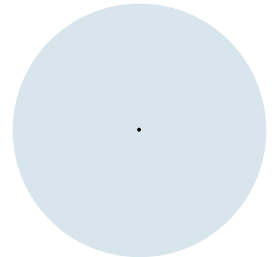
- Typprüfung mit **instanceof**

```
if (variable instanceof <Typname>) { ... }
```

Abstrakte Klassen

- Definiert durch `public abstract class <Klassenname> { ... }`
- können, ähnlich wie Interfaces, Methoden ohne Rumpf definieren (**abstrakte Methoden**)
`public abstract <Rückgabetyp> methodName(<Parameter>);`
- **Vorlage** für Implementierung
- **Abstrakte Klasse**
 - Abstrakte Klassen können **nicht instanziiert** werden
 - Eine konkrete Klasse, die von einer abstrakten Klasse erbt, muss alle abstrakten Methoden überschreiben und implementieren.
- **Anders als bei Interfaces**
 - Abstrakte Klasse: **Implementierung** von Methoden **möglich**
Interface: Alle Methoden **ohne Implementierung** (Ausnahme: default-Methoden ab Java 8, nicht Teil dieser VL)
 - Abstrakte Klasse: **Attribute** erlaubt
Interface: **Keine Attribute** erlaubt

Interfaces vs. Vererbung



Interfaces

- Gut geeignet für die Modellierung bei Vorhandensein **gleicher Schnittstellen**
- Gut geeignet, wenn Interface ganz **unterschiedlich implementiert** werden kann, wenn also Schnittstellen gleich sind, **Funktionalität** aber **verschieden** ist
- Beispiel:
 - `interface Drawable { void draw(); }`
`class Circle implements Drawable`
`class Rectangle implements Drawable`
 - Die Methode `draw()` wird je nach geometrischer Form ganz unterschiedlich implementiert. Eine Ausgabe (Bildschirm/Drucker) kann diese Methode dann verwenden. Durch das Interface ist die Signatur vorgegeben.